

Software Lifecycle Management



Fachbereich Elektrotechnik
und Technische Informatik

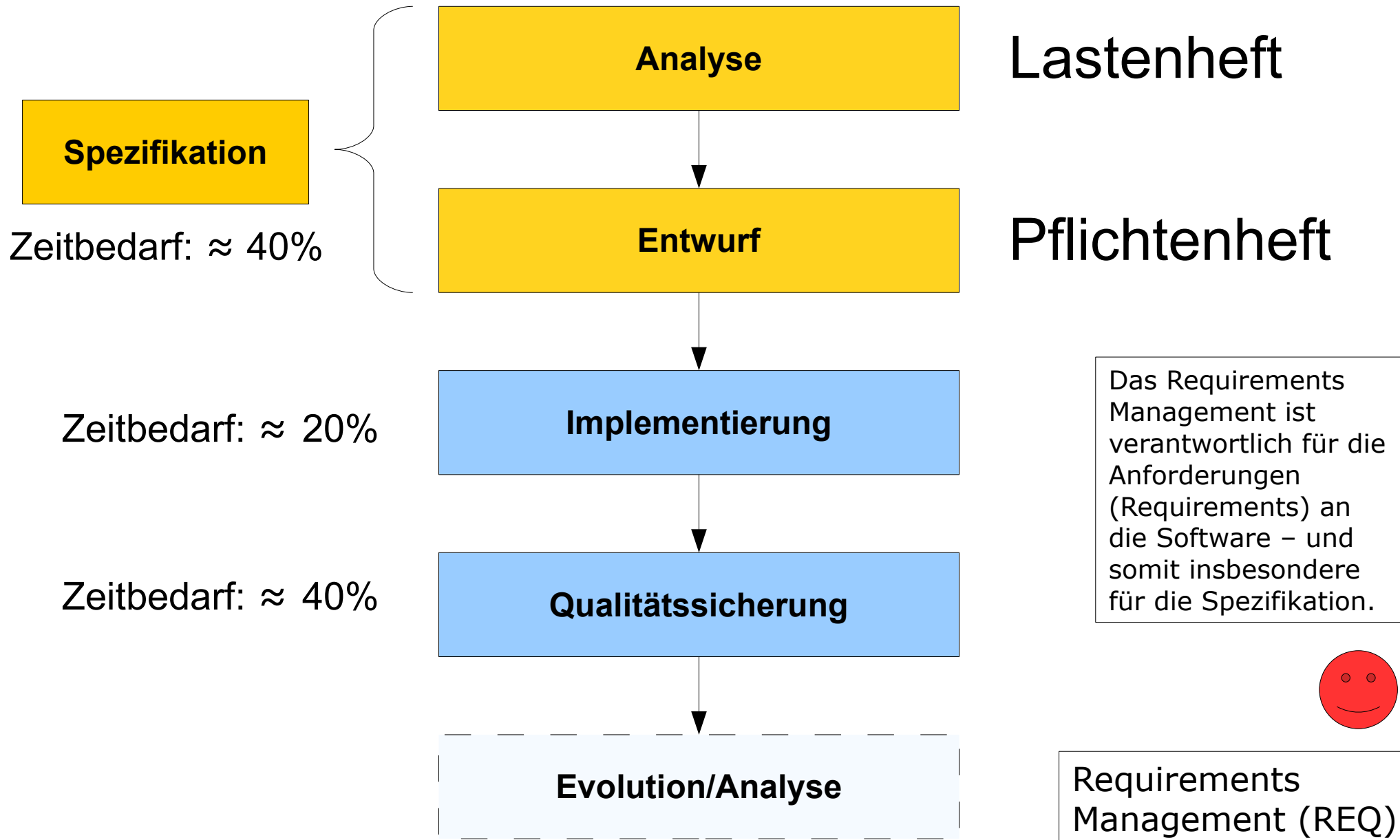
Dr. rer. nat. Nils Beckmann
nils.beckmann@th-owl.de

Spezifikation

Agenda

- Requirements Management (REQ)
 - Spezifikation: Lastenheft und Pflichtenheft
 - Erstellung, Strukturen, Inhalte, Nutzer, Verwendung, Sprache, Detailreichtum
 - Funktionale Einheiten
 - *use-cases* (Anwendungsfälle)
 - *user-stories* (Anwendererzählung)
 - EVA-Prinzip

Erinnerung: Softwareprozessphasen



Lastenheft und Pflichtenheft

	Lastenheft	Pflichtenheft
Wird erstellt von...	Kunde, Nicht-Fachmann, Auftraggeber ggf. in Zusammenarbeit mit Vertriebler, Requirements Engineer, Auftragnehmer, etc.	Auftragnehmer, Requirements Engineer, Software-Architekten, Software-Entwickler, etc.
Dient als Grundlage für...	Ausschreibung, (Unterstützung für) Pflichtenheft	Angebot, Vertrag
Nutzer	u.A. Kunde, Manager, Vertriebler, Architekten, etc.	u.A. Kunden, Manager, Vertriebler, Entwickler, Architekten, Warter, etc.
Beantwortet	Was?	Wie? Womit?
Sprache	Eher ohne Fachbegriffe	Fachsprache
Grundlage	Ideen, Gespräche, etc.	Lastenheft

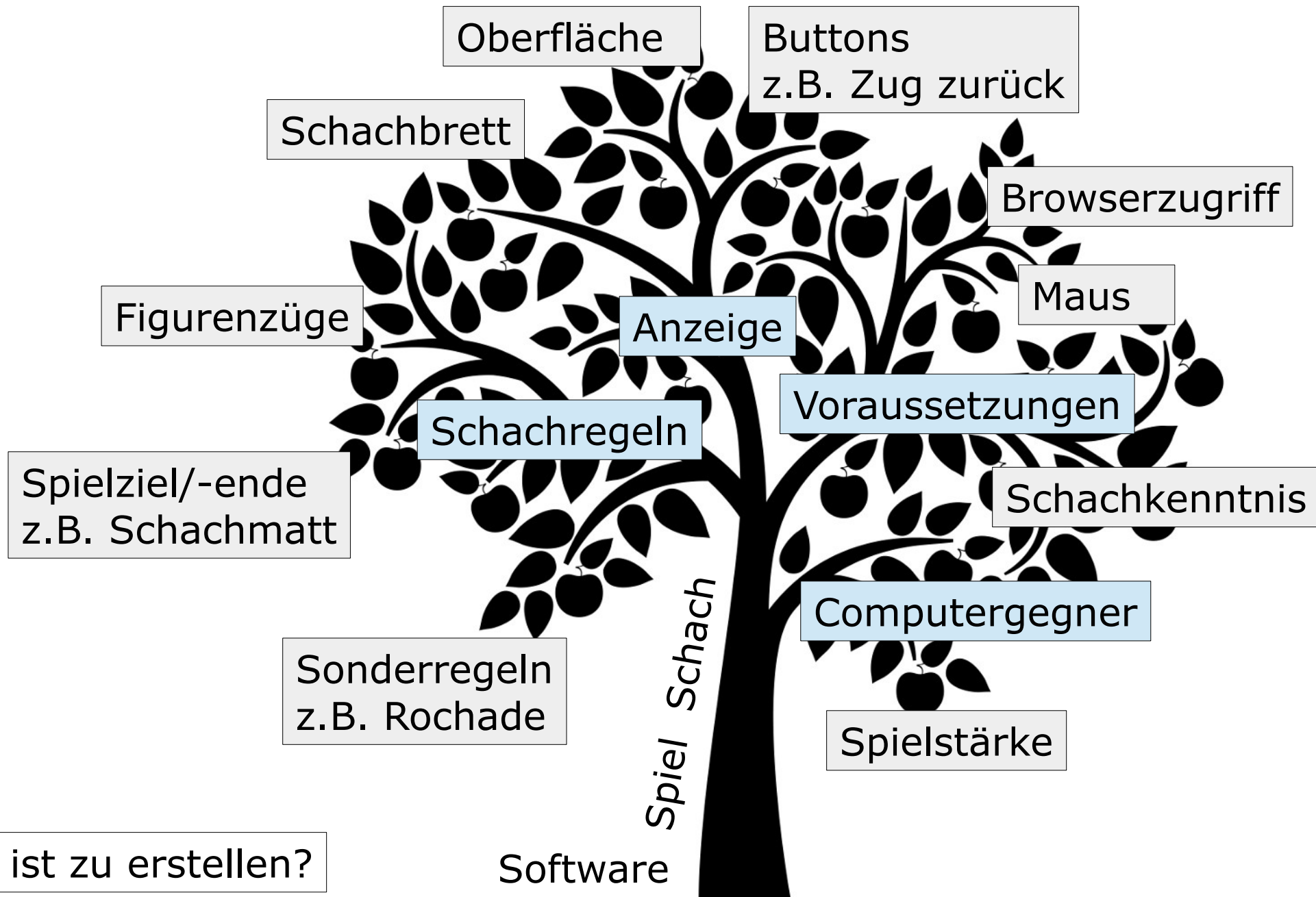
Erstellung der SAS (Klassisch)

- Zunächst wird das Lastenheft erstellt mit den Anforderungen an die Software
- Danach wird das Pflichtenheft aus dem Lastenheft entwickelt
 - Das Pflichtenheft stellt insbesondere eine **technische Verfeinerung** und Detaillierung des Lastenhefts dar
 - Große Teile des Lastenhefts können i.d.R. ins Pflichtenheft übernommen werden
- Die **SAS** dient als **Grundlage für die Implementierung**

Anspruch an die SAS

- Das Lastenheft als auch das Pflichtenheft sollen die 9 Kriterien für **gute Anforderungen** erfüllen (siehe Vorlesung 102-Anforderungen)
- Im Idealfall erlaubt die vollständige Spezifikation (SAS) **keine/wenige Freiheiten** mehr während der Implementierung, es müssen dort also keine (großen) Entscheidungen mehr getroffen werden
 - Würde man die SAS also 10 verschiedenen fehlerfrei arbeitenden Entwicklerteams geben, würde theoretisch 10x dieselbe Software entstehen
- Das Lastenheft soll die Software **1:1 abbilden** in Textform (inklusive Diagrammen, Tabellen, usw.); das Pflichtenheft spezifiziert dies dann technisch

Baum als Metapher zum Lastenheft

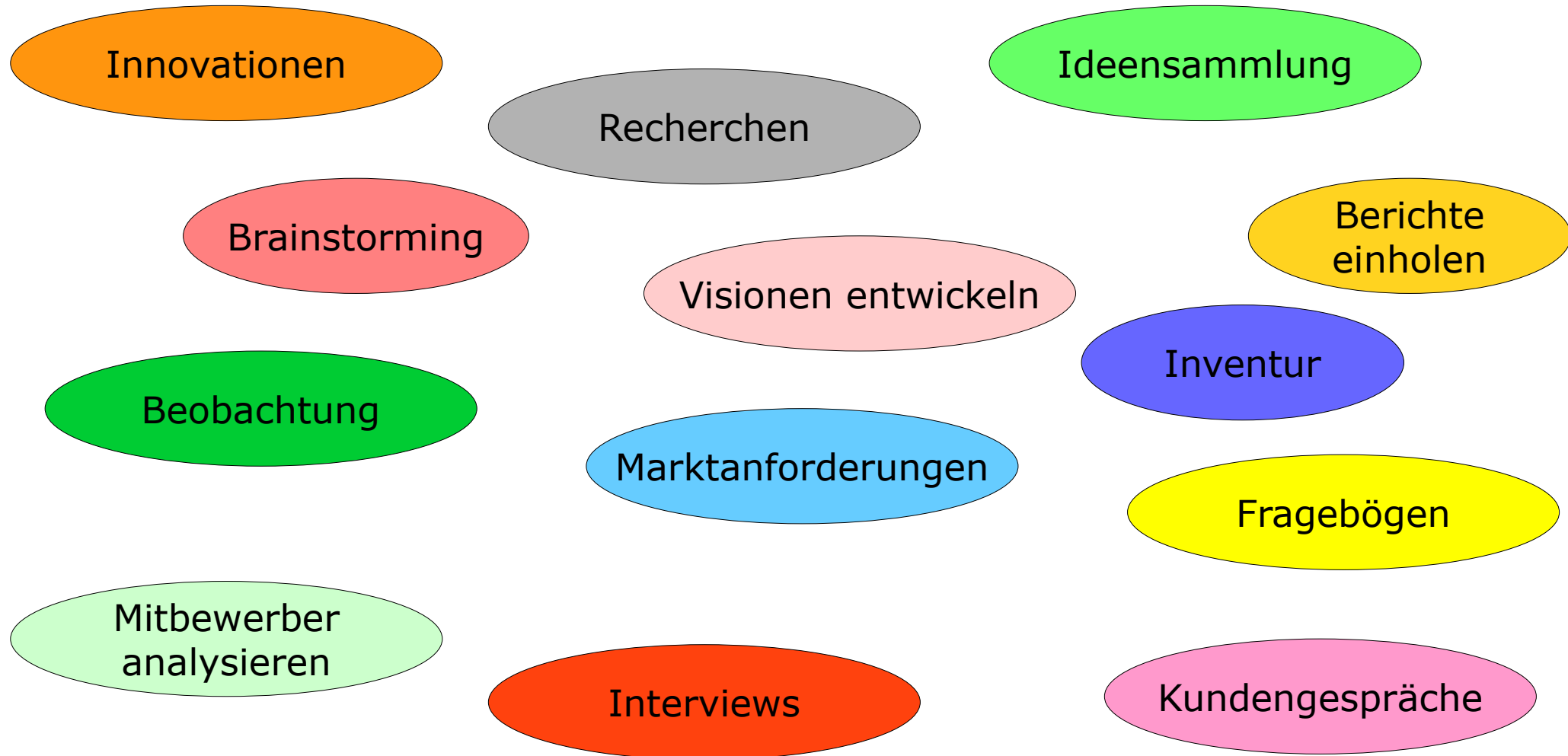


Analogie zum Hausbau

- Plan: "Baue mir ein schönes Haus."
 - Mit diesem einen Satz allein bleiben viele Fragen unbeantwortet.
- Lastenheft (Details der Ausgestaltung):
 - Wie viele Etagen soll das Haus haben? Wo soll es stehen? Anzahl, Form, Anordnung Fenster? Anzahl, Schnitt, Struktur der Räume? Deckenhöhe? Keller? Wanddicke? Garten (m², Bäume, Hecken, Rasen, etc.)? Bodenbelag (Fliesen, Dielen, Vinyl, etc.)? Veranda, Balkon, Terrasse? Dachform und Bedeckung? Welche Art von Haus (Villa, Einfamilienhaus, Reihenhaushaus, Wald-/Berghütte, Ferienhaus, Mehrparteienhaus, usw.)? Welche Anschlüsse vorhanden (Strom, Gas, Wasser, Wärme, Steckdosen, usw.)? Welche Extras (Solaranlage, Loft, Dachterrasse, Schrägen, Auslassungen, usw.)? Wieviel Zeit und Kosten sind eingeplant? usw.
- Pflichtenheft (Technische Details):
 - Welche Betonart? Kabeldicken? Heizungsart und -technik (Gas, Öl, Wärmepumpe, Fernwärme, Solarthermie, usw.)? Plan der Verlegung von Kabeln, Rohren, Dämmungen, Abdeckungen, Dichtungen, usw.? Materialien für Böden, Wände, Fassaden, Ziegel, Fenster, usw.? Wie werden Fliesen, Böden, usw. befestigt bzw. geklebt? Welche Hersteller, Lieferanten, Gewerke, Märkte werden gewählt? usw.

Methoden der Anforderungserhebung

- Unterstützend bei der Generierung des Lastenhefts:



Struktur des Lastenhefts

1) Zielbestimmung

➔ Welche Ziele sollen durch den Produkteinsatz erreicht werden?

2) Produkteinsatz

➔ Welche Anwendungsbereiche, Einsatzanforderungen und Zielgruppen gibt es?

Struktur des Lastenhefts

3) Produktfunktionen

- ➔ Auflistung der Kernfunktionen (Epics & Features bzw. Produktanforderungen), also funktionale Anforderungen
- ➔ Die Auflistung kann hier ohne sekundäre Funktionen (Systemanforderungen, Untieranforderungen) geschehen und ohne Details

4) Produktdaten

- ➔ Welche Daten werden gespeichert und übertragen (Netzwerk) durch das Produkt?

Struktur des Lastenhefts

5) Produktleistungen

- ➔ Nicht-funktionale Anforderungen an die Funktionen (funktionalen Anforderungen) bezüglich Zeiten, Umfang, Genauigkeit, etc.

6) Qualitätsanforderungen

- ➔ Sonstige (nicht-funktionale) Anforderungen bezüglich Zuverlässigkeit, guter Benutzbarkeit, Effizienz, etc. an die Software

7) Ergänzungen

- ➔ Sonstiges Hinweise, zum Beispiel außergewöhnliche Anforderungen, bestimmte Risiken, wichtige Literaturhinweise, spezielle Details oder Grafiken, usw.

Struktur des Pflichtenhefts (I)

1) Zielbestimmung

- a) Muss-Kriterien (obligatorisch) (*must have*)
- b) Wunsch-Kriterien (optional) (*can have*)
- c) Abgrenzungskriterien (nicht geplant) (*boundaries*)

2) Produkteinsatz

- a) Anwendungsbereiche
- b) Zielgruppen

→ z.B. auch mit Hinweisen auf fachliche Voraussetzungen für die Bedienung der Software, Nutzervorbildung, mögliche Hürden, usw.

c) Betriebsbedingungen

→ Physikalische Umgebung, Betriebszeit, Beaufsichtigung, etc.

Struktur des Pflichtenhefts (I)

3) Produktumgebung

- a) Software

- b) Hardware

- c) Orgware

 - Alle organisatorischen Randbedingungen

- d) Produkt-Schnittstellen

 - zu Software, Hardware, Netzwerk, Internet, etc.

Struktur des Pflichtenhefts (I)

4) Produkt-Funktionen

a) Funktion 1

b) Funktion 2

c) etc.

→ Was kann man mit dem Produkt tun? Aus Benutzersicht, unabhängig von Bildschirm-Layout oder Tastenbelegungen, etc.

5) Produkt-Daten

a) Daten 1

b) Daten 2

c) etc.

→ Welche (Arten von) Daten werden gespeichert bzw. übertragen, in welchem Umfang, usw.

Struktur des Pflichtenhefts (I)

6) Produkt-Leistungen

→ Zeitbezogene, umfangbezogene Zusatzanforderungen an Funktionen

7) Benutzungsoberfläche

→ Layout von Bildschirm und Druck, Dialoge, GUI, UX, Design, etc.

8) Qualitäts-Zielbestimmung

→ Nicht-Funktionale Anforderungen an Zuverlässigkeit, gute Benutzbarkeit, etc.

9) Globale Testszenarien und Testfälle

Struktur des Pflichtenhefts (I)

10) Entwicklungsumgebung

- ➔ Was wird für die Entwicklung des Produkts (zwingend) benötigt:
Hardware, Software, Orgware, Compiler, IDE's, Lizenzen, Zugriffe auf
(Entwicklungs-)Schnittstellen, Prototypen, usw.

11) Ergänzungen

- ➔ z.B. Installationsbedingungen, Bereitstellung von Testdaten oder
Hardware oder Hilfspersonal, bauliche und räumliche
Voraussetzungen, etc.

Struktur des Pflichtenhefts (II)

1) Einleitung (*introduction*)

a) Ziel des Anforderungsdokuments (*purpose*)

→ Wofür ist das Dokument da, nicht die Software

b) Anwendungsbereich des Produkts (*scope*)

→ Wofür ist die Software da und wo ist geplant sie einzusetzen?

c) Definitionen, Akronyme und Abkürzungen (*definitions*)

d) Referenzen (*references*)

→ Quellenangaben, Links, usw. (kann z.B. auch ans Ende des PH)

e) Überblick über den Rest des Dokuments (*overview*)

→ Kurze Beschreibung der folgenden Inhalte

Struktur des Pflichtenhefts (II)

2) Allgemeine Beschreibung (*description*)

a) Produktperspektive (*perspective*)

➔ Vergangenheit (Was gab es?), Gegenwart (Was ist vorhanden?), Zukunft (Was wird sein? Was ist geplant?); gibt es Informationen zu Kommerzialisierung, Lizenzmodellen, Visionen, usw.

b) Produktfunktionen (*functions*)

➔ Allgemeine Beschreibung der großen Produktfunktionen, was man damit tun kann

c) Benutzercharakteristika (*characteristics*)

➔ Benutzergruppen (Vorwissen, wie sie die Software nutzen können, deren Ziele und Bedarfe, usw.)

d) Allgemeine Beschränkungen (*constraints*)

e) Voraussetzungen und Abhängigkeiten (*dependencies*)

Struktur des Pflichtenhefts (II)

3) Spezifische Anforderungen (*requirements*)

a) Funktionale Anforderungen (*functional requirements*)

b) Nicht-Funktionale Anforderungen (*non-functional requirements*)

c) Schnittstellen (*interfaces*)

➔ Software, Hardware, Netzwerk, Datenbanken, etc.

◆ + ggf. weitere Unterkapitel: z.B. Entwurfseinschränkungen (*constraints*), usw., je nach Softwareprodukt

4) Anhänge (*appendices*)

5) Index (*index*)

Strukturen für die SAS

- "Software-Engineering", Sommerville, 2007, S. 169
- "Lehrbuch der Software-Technik", Balzert, 1996, S. 59, S. 105 ff.
- IEEE/ANSI Standard 830-1996
- DIN 66272 ISO/IEC 9126 -> DIN ISO/IEC 25000
<https://www.din.de/de/mitwirken/normenausschuesse/nia/normen/wdc-beuth:din21:204260933>
- Capability Maturity Model Integration (CMMI): CMMI Product Team: CMMI® for Development, Version 1.2, Improving processes for better products. Hrsg.: Software Engineering Institute, Carnegie Mellon. Pittsburgh, Pennsylvania 2006.
- Volere-Modell: <http://www.volere.co.uk/books.htm>
- Im SLM-Ordner "Vorlagen" im ILIAS liegen weitere, angepasste Vorlagen für Lastenheft und Pflichtenheft
- usw.

Wer nutzt die SAS?

- **Systemkunden**

- Legen Anforderungen im Lastenheft fest, die dann ins Pflichtenheft übertragen werden.
- Lesen das Pflichtenheft um zu überprüfen, dass alle Anforderungen ihren Erfordernissen entsprechen.
- Nehmen das Pflichtenheft ab, das Grundlage für einen Vertrag sein können sollte.

- **Vertriebler**

- Verwenden das Pflichtenheft zum Erstellen des Angebots für das System.

Wer nutzt die SAS?

- **Manager**
 - Planen gemäß dem Pflichtenheft den Entwicklungsprozess und verplanen Ressourcen.
- **Systementwickler**
 - Benutzen die Anforderungen um zu verstehen, was für ein System entwickelt werden soll.
- **Systemtester**
 - Benutzen die Anforderungen um Validierungstests für das System zu entwickeln.
- **Systemwarter**
 - Verwenden die Anforderungen um das System und seinen Wechselwirkungen besser zu verstehen.

Vorgehensweise Erstellung

- Die Erstellung von Lastenheft und Pflichtenheft ist meist ein **iterativer** Prozess.
- Verschiedene **Blickwinkel** verschiedener Personen ermöglichen die Verfeinerung und Qualitätserhöhung der SAS.

Dateien:

...

...

20180417_Pflichtenheft_Karls_V5.odt

20180420_Pflichtenheft_Alteheld_V6.odt

20180422_Pflichtenheft_Bruckner_V7.odt

20180423_Pflichtenheft_Meier_V8.odt

20180425_Pflichtenheft_Beckmann_V9.od

t

Wahl der Sprache

- Die verwendete Sprache sollte (insbesondere im Pflichtenheft) Fachsprache sein, d.h.:
 - Es werden die adäquaten Fachbegriffe verwendet
 - Es wird klar und präzise formuliert
 - Unnötige Füllwörter werden vermieden

Allgemeine sprachliche Hinweise

- Jeder Satz für sich alleinstehend sollte eine **eigene Aussage** beinhalten
- Lieber **kurze Sätze** verwenden, Bandwurmsätze vermeiden
- Sagen, was man sagen will
- Umschreibungen vermeiden
- Technische Themen sind oft für sich allein schon kompliziert genug:
 - Zum besseren Verständnis: Einfache und klare Sprache – das Thema sollte nicht durch die Sprache noch komplizierter werden als es eigentlich ist

Füllwörter

- Füllwörter fügen dem Satz i.d.R. keinen weiteren Informationsgehalt hinzu
 - Für Romane u.a. (Epik, Dramatik, Lyrik) sind diese zur Unterhaltung essentiell
 - In einer Spezifikation sollten sie vermieden werden
- Überflüssige: "sowieso", "sicherlich", "eigentlich", "jedenfall", ...
- Übertreibungen: "enorm", "sehr", "drastisch", ...
- Emotionales und Dramatisierend: "schade", "schlimm", "enttäuschend", "furchtbar", "leider", ...

Funktionale Einheiten

- Die Summe aller **Funktionalen Einheiten** (Anwendungsfälle) der Software macht die **Funktionalen Anforderungen** aus.
- Die formulierten Anforderungen können in der SAS in funktionalen Einheiten **strukturiert** und **ausformuliert** werden.
- Die Funktionalen Anforderungen können dabei einzelne Funktionen beschreiben oder Funktionskomplexe (Sammlung von ähnlichen, einzelnen Funktionen). Dies kann u.A. über verschiedene ALM-Werkzeuge geschehen:
 - Anwendungsfälle (*use-cases*)
 - Anwendererzählungen (*user-stories*)

Anwendungsfall

- Ein **Anwendungsfall** (*use case*) ist eine Zusammenfassung für die Durchführung einer Aktion mit einem bestimmten (Software-)System:
 - Akteur
 - Aktion
 - (Vollständige) Liste der Einzelschritte
 - System
 - Ziel (der Aktion)

Definition Akteur/Akteur

- Ein **Aktor** agiert mit dem System und kann Anwendungsfälle auslösen und durchführen.
 - Akteure können Personen (Nutzer des Systems), über Schnittstellen verbundenen externe Geräte oder mit dem System verbundene Nachbarsysteme sein.

Beispiel Anwendungsfall

- Anwendungsfall:

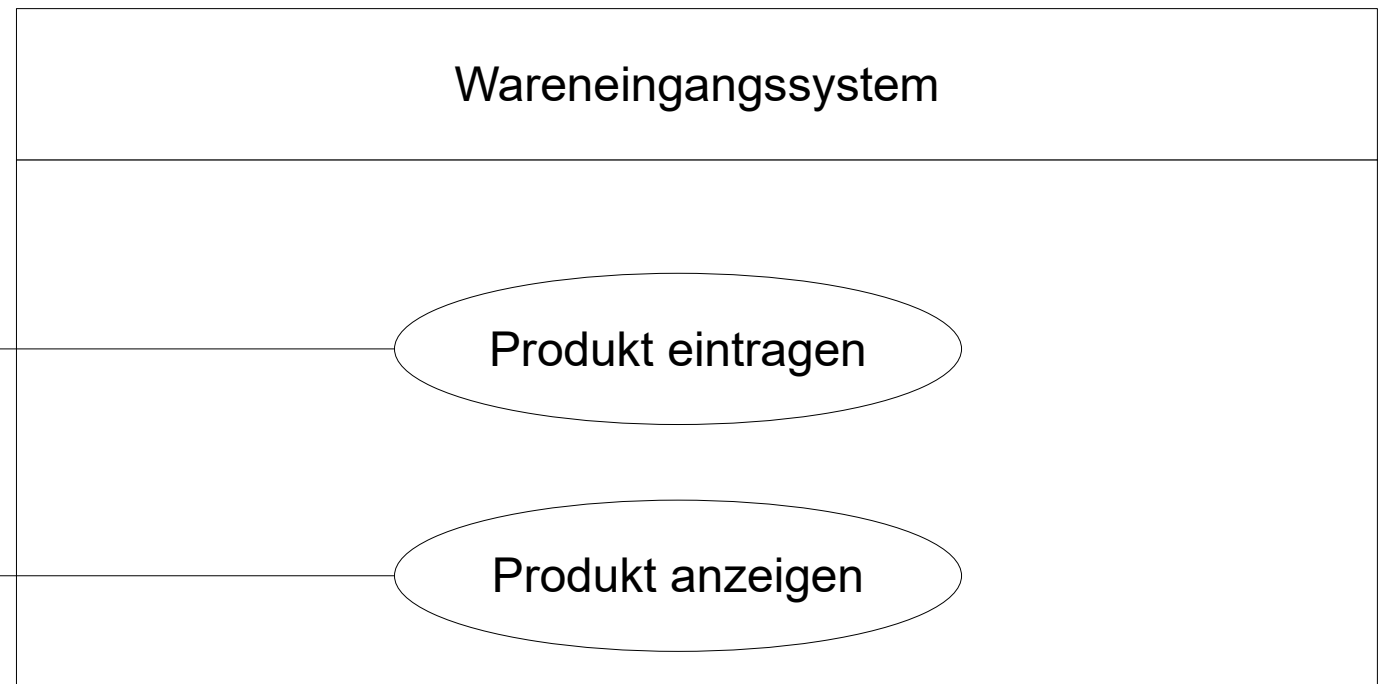
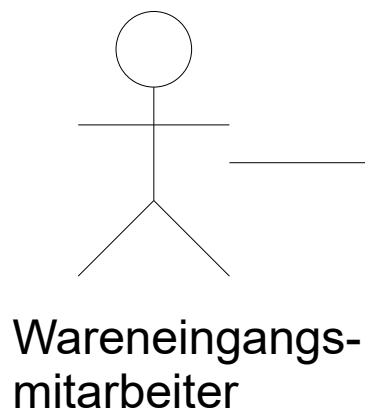
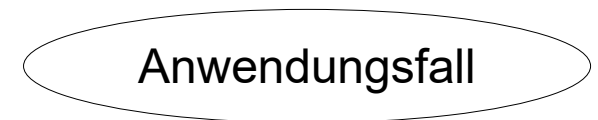
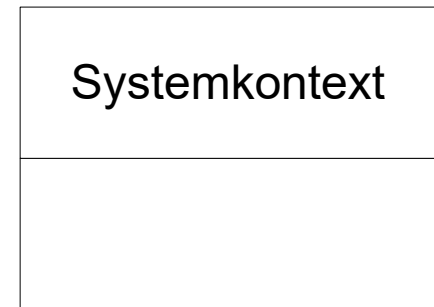
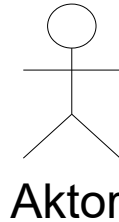
- Der Mitarbeiter am Wareneingang hat das Ziel eine Ware einzubuchen. Dazu sind die folgenden Schritte notwendig:
 - Ware wird geliefert
 - Ware wird angenommen
 - Mitarbeiter unterzeichnet auf dem Lieferdokument den Erhalt der Ware
 - Mitarbeiter scannt die Waren-ID mit dem Barcode-Scanner ein
 - Mitarbeiter prüft in der Computersoftware, ob die Warenbeschreibung mit der gelieferten Ware übereinstimmt
 - Stimmt überein: Mitarbeiter bestätigt Übereinstimmung in der Computersoftware
 - Stimmt nicht überein: Mitarbeiter bestätigt Abweichung in der Computersoftware, dokumentiert diese Abweichung schriftlich und gibt entsprechende digitale Meldung über Erstellung eines Tickets an Vertriebsdienst weiter

Grundlagen

UML-Anwendungsfalldiagramm

- Bausteine:

- Akteur/Akteur
- (System-)Kontext + (System-)Grenzen
- Anwendungsfälle (Use Cases)



Tools: UML-Diagramme

- **Frei verfügbar:**

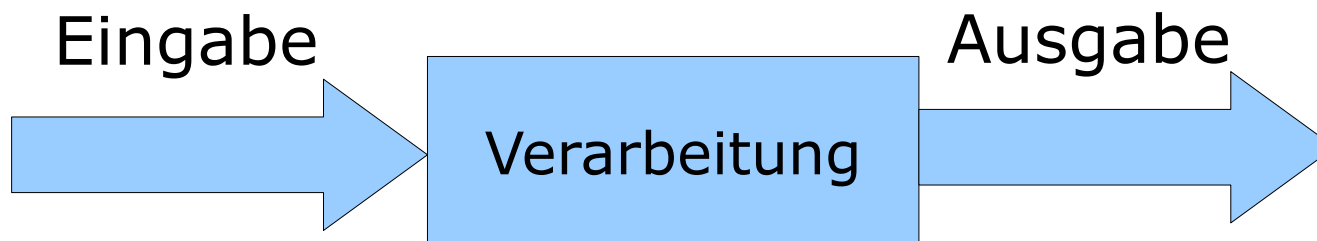
- Argo UML
<http://argouml.tigris.org/>
- Umlet
<http://www.umlet.com/>
- u. v. m. (Lange Liste von Tools auf Wikipedia -->)
https://en.wikipedia.org/wiki/List_of_Unified_Modeling_Language_tools

- **Kommerziell:**

- Visual Paradigm
<https://www.visual-paradigm.com/download/community.jsp>
- Microsoft Visio
<https://products.office.com/de-DE/visio?ms.officeurl=visio>
- u. v. m.

Das EVA-Prinzip

- Das EVA-(Grund-)Prinzip der Datenverarbeitung:
- E = Eingabe
- V = Verarbeitung
- A = Ausgabe



Anwendungsfälle erweitert nach dem EVA-Prinzip

- **Titel:** Artikel eintragen
- **Kurzbeschreibung:** Der Wareneingangsmitarbeiter trägt den Produktnamen und Stückzahl ein und legt einen Wareneintrag an.
- **Aktor:** Wareneingangsmitarbeiter
- **Vorbedingungen:** Es muss noch ein freier Platz im Warenlager sein
- **Ablaufbeschreibung:** Prüfe, ob noch Platz im Warenlager ist. Falls kein Platz mehr im Warenlager ist, zeige eine Fehlermeldung an. Falls noch Platz ist, prüfe, ob das Produkt schon vorhanden ist. Falls ja, addiere zur Stückzahl des Produkts die Stückzahl des Neueingangs. Falls nein, lege einen neuen Produkteintrag mit der Stückzahl an. Zeige dann eine Bestätigungsmeldung für den Aktor.
- **Auswirkungen:** Das Produkt ist eingetragen mit entsprechender Stückzahl oder es wird eine Fehlermeldung angezeigt.
- **Anmerkungen:** In der Realwelt sollte das neue Produkt auch tatsächlich existent sein – dies prüft die Software jedoch nicht.

User-Stories

- Eine in Alltagssprache formulierte Software-Anforderung.
 - Maximal 1-2 Sätze lang.
 - Verzichtet bewusst auf Fachbegriffe.
 - Stellt eine Benutzeranforderung dar.
- Satzvorlage für eine User-Story:

Als <Rolle> möchte ich <Aktion> damit <Ziel>.

- Beispieltext:
 - „Als ein Mitarbeiter im Wareneingang möchte ich Lieferungen in Kistenform annehmen und im digitalen System über einen Barcode-Scanner verbuchen, so dass jede gelieferte Kiste identifiziert und digital erfasst ist.“

Begriffsdifferenzierungen

- Eine **User-Story** beschreibt einen Vorgang sehr prägnant und durchaus umgangssprachlich aus der Sicht eines Nutzers/Kunden und rollenspezifisch. Damit ist dies eine Benutzeranforderung. Wird häufiger in der agilen Entwicklung verwendet.
- Ein **Use-Case** ist deutlich spezieller und technischer und umfangreicher. Er bezieht sich auf einen bestimmten Arbeitsablauf innerhalb der Softwareanwendung. Ist damit eine funktionale Anforderung.
- Ein **Feature** beschreibt auf einer höheren technischen Ebene ein Funktionalitätspaket der Software und kann diverse Use-Cases beinhalten, muss aber nicht rollenspezifisch sein.
- I.d.R. gibt es mehr Use-Cases als Features oder User-Stories.

Anforderungen Klassisch/Agil

- **Klassisch**

- „Requirements-Breakdown“

- Business-Anforderungen

- Produkt-Anforderungen

- System-Anforderungen

- Komponenten-Anforderungen

- Hauptanforderung

- Unteranforderung 1

- Unteranforderung 2

- ...

✓ (eher) Use Cases

- **Agil**

- Portfolio

- Epics

- Features

- Backlogs/Tasks

✓ (eher) User Stories

Zusammenfassung

- Die **Spezifikation** umfasst Analyse und Enturf
- Ein wichtiges Ergebnis der Spezifikation ist die **Softwareanforderungsspezifikation** (SAS), sie...
 - ...umfasst Lastenheft und Pflichtenheft
 - ...beschreibt eine Software vollumfänglich
 - ...ist das zentrale Dokument, das vor einer Implementierung erstellt wird
- **Anforderungen** können in der SAS weiterentwickelt werden zu **Funktionalen Einheiten**, welche aus **Anwendungsfällen** (use case) und **Anwendergeschichten** (user story) bestehen können

Fragen?